In many cases, you will have a sizable number of files to score against, and for this you might want to leverage the distributed capacity of Apache Spark. We create a Pandas UDF to score our model across all of the labels. Pay attention to *Figure 6.34*, where we use the broadcast option. Broadcast is a functionality of Apache Spark that would copy the object across all nodes. Why do we need a broadcast? It is not a requirement, but the way Pandas UDF works is that they will have to load your model in the memory of each node each time a chunk of data arrives. Broadcasting your model can accelerate that process. Consider the size of your model and your node memory capacity before doing this for any other use case.

```
@pandas_udf("struct< label: int, labelName: string>")
def apply_cv(images_iter: Iterator[pd.Series]) -> Iterator[pd.DataFrame]:

    model = model_b.value.to(torch.device("cuda"))
    model.eval()

    id2label = {0: 'buildings',
                1: 'forest',
                2: 'glacier',
                3: 'mountain',
                4: 'sea',
                5: 'street'}

    with torch.set_grad_enabled(False):
        for images in images_iter:
            pil_images = torch.stack(
                [
                    feature_extractor(b)
                    for b in images
                ]
            )
            pil_images = pil_images.to(torch.device("cuda"))
            outputs = model(pil_images)

            preds = torch.max(outputs, 1)[1].tolist()
            probs = torch.nn.functional.softmax(outputs, dim=-1)[:, 1].tolist()

            yield pd.DataFrame(
                [
                    {"label": pred, "labelName":id2label[pred]} for pred in preds
                ]
            )
```

Figure 6.35 – Creating a Pandas UDF with an Iterator to yield results across chunks